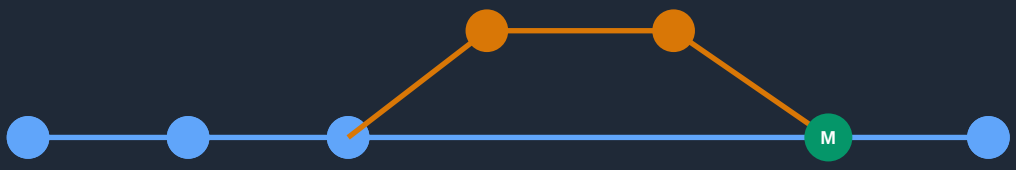
Git & GitHub

Complete Cheat Sheet

Concepts with diagrams, then every command with explanations
Course: 26_git_github | Beginner to professional workflow



Developed by Sheharyar Sarmad

LinkedIn: https://www.linkedin.com/in/sheharyar-sarmad-9b7736289/
GitHub: https://github.com/Sheharyar-Sarmad

Contents

Part 1 - Concepts (visual)

- 1. What is Git? Why use it?
- 2. A short history of Git
- 3. Centralized vs distributed version control
- 4. Git vs GitHub
- 5. Installing and configuring Git
- 6. How Git works: the three areas
- 7. File lifecycle
- 8. Git objects and commits
- 9. .gitignore and .gitkeep
- 10. Branching and merging
- 11. Merge conflicts
- 12. Stashing and tagging
- 13. Rebase basics
- 14. Remotes: fetch, pull, push
- 15. GitHub, GitHub Desktop and VS Code
- 16. The modern workflow (pull requests)

Part 2 - Command reference

- Setup | Start a repo | Inspect | Stage & commit | Undo | Branches | Merge & conflicts | Stash | Tags | Rebase | Remotes | VS Code / GitHub Desktop equivalents

Part 3 - Quick workflows

- Daily workflow | Full PR workflow | Troubleshooting | Final checklist

PART 1 - CONCEPTS

1. What is Git?

Git is a free, open-source **version control system**. It runs on your computer and records snapshots of your project over time, so you can see what changed, who changed it, go back to any earlier version, and work on new ideas without breaking working code.

- **History**: every saved version (commit) is kept.
- **Undo safely**: return to an earlier state when something breaks.
- **Teamwork**: many people can work on one project at the same time.
- **Experiments**: try ideas on separate branches.

2. A short history of Git

Linus Torvalds created Git in **2005** to manage the Linux kernel after the tool they previously used became unavailable. Goals: speed, simple design, strong support for parallel branches, and fully distributed work. **GitHub** launched in 2008 as a website to host Git repositories, and was acquired by Microsoft in 2018.

3. Centralized vs distributed version control

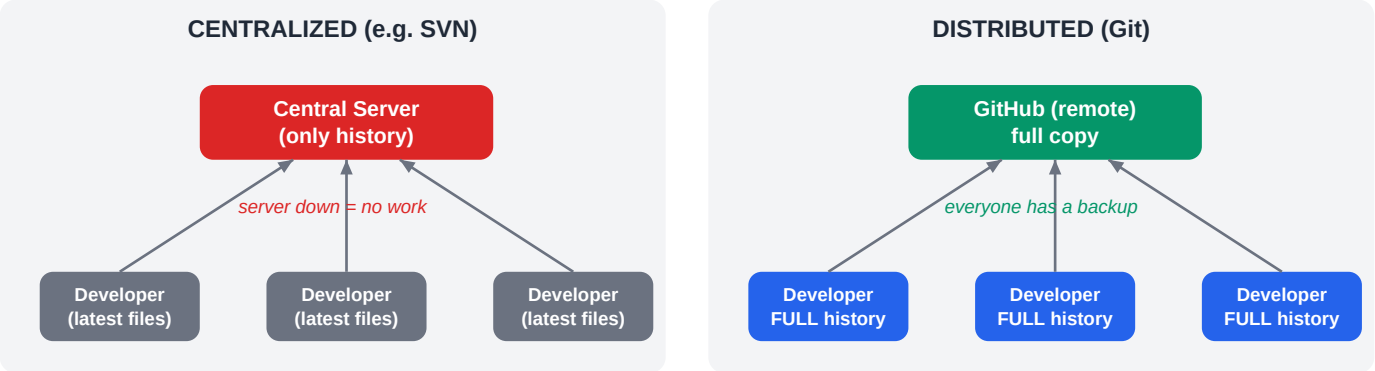


Figure 1 - Centralized systems depend on one server. In Git, every developer holds the full history.

4. Git vs GitHub

Name	Meaning
`Git`	A tool installed on your computer. Tracks changes. Works offline.
`GitHub`	A website (github.com) that hosts Git repositories online for backup, sharing, pull requests and collaboration.
`Alternatives`	GitLab and Bitbucket do a similar job to GitHub. Git works with all of them.

> **Analogy:** Git is like the save-history feature in your editor. GitHub is like Google Drive for that history, plus tools to review each other's work.

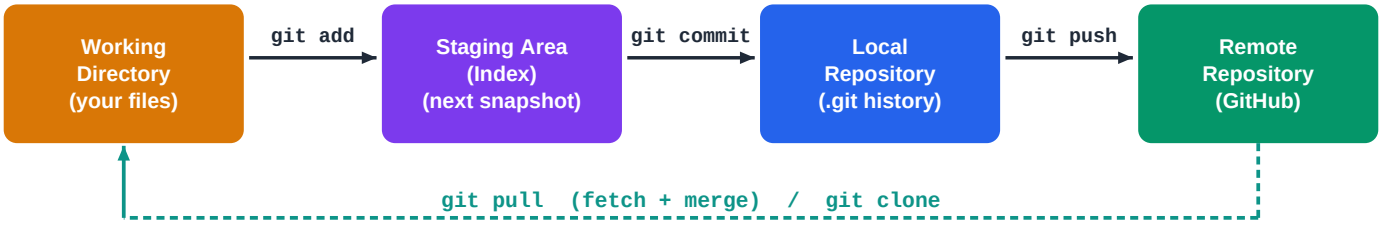
5. Installing and configuring Git

Download Git from **git-scm.com** (Windows installs **Git Bash** too). Check the install, then tell Git who you are. This is needed once per computer.

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --list
```

Create a GitHub account: go to **github.com**, click **Sign up**, enter email, password and username, then verify your email. Use the same email in `user.email` so commits are linked to your profile.

6. How Git works: the three areas

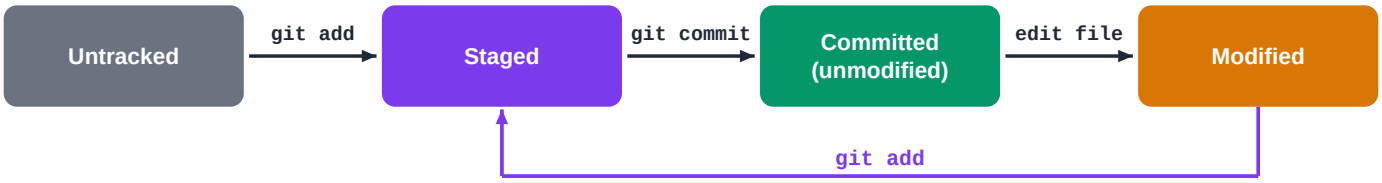


git restore <file> discards edits in the working directory

Figure 2 - Changes move left to right: edit, stage, commit, push.

- **Working Directory**: the real files you edit.
- **Staging Area**: a waiting room where you choose what goes into the next commit.
- **Local Repository**: the permanent history in the hidden `.git` folder.
- **Remote**: a copy of the repository on GitHub.

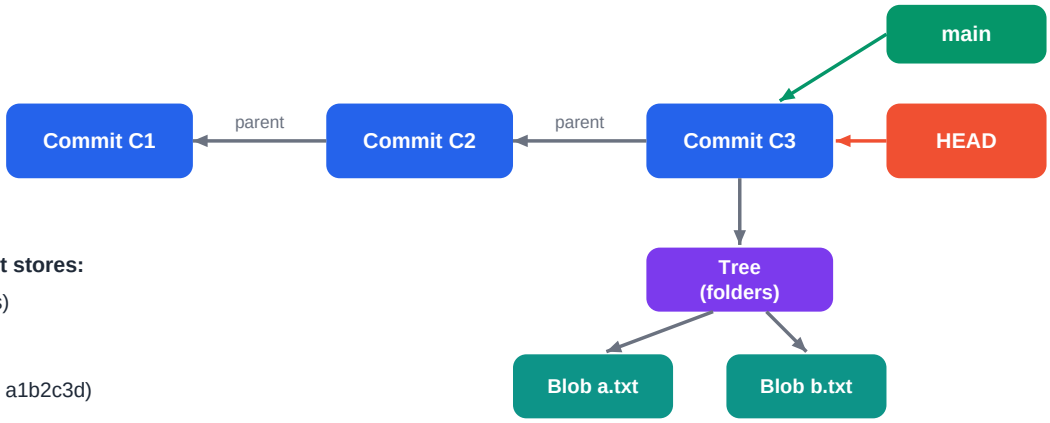
7. File lifecycle



git status tells you which state every file is in

Figure 3 - Every file is untracked, staged, committed (unmodified) or modified.

8. Git objects and commits



Each commit stores:

- a snapshot (tree + blobs)
- author, date, message
- pointer to its parent
- a unique SHA-1 ID (e.g. a1b2c3d)

Figure 4 - Commits form a chain through parent pointers. HEAD points to the branch you are on.

- **Blob**: the content of one file.
- **Tree**: a folder listing that points to blobs and other trees.
- **Commit**: a snapshot (tree) plus author, message and parent.
- **Branch**: a name that points to a commit. **HEAD**: points to your current branch.

9. .gitignore and .gitkeep

`.gitignore` lists files Git should **never track**: secrets, build output, dependencies. `.gitkeep` is just an empty file by convention so Git can keep an **empty folder** (Git only tracks files, not folders).

```
# .gitignore
node_modules/
.env
*.log
dist/

# keep an empty folder
touch logs/.gitkeep
```

> **Note:** .gitignore only affects untracked files. If a file is already committed, run `git rm --cached <file>` to stop tracking it.

10. Branching and merging

A branch is just a movable pointer to a commit. Branching is cheap and safe.

`git switch -c feature-login` ... `git merge feature-login`

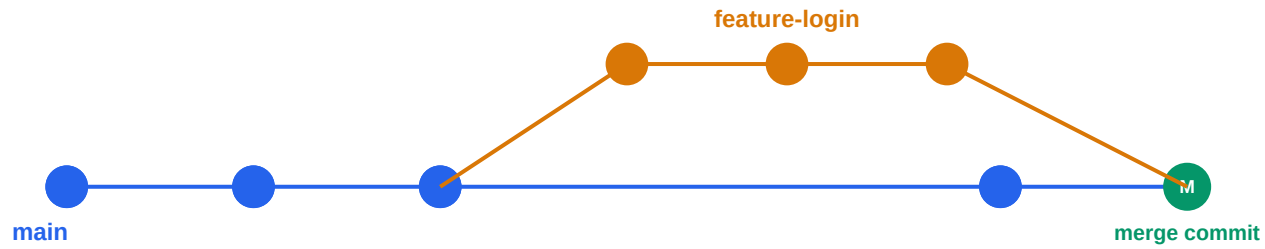


Figure 5 - Work on a feature branch, then merge it back into main.

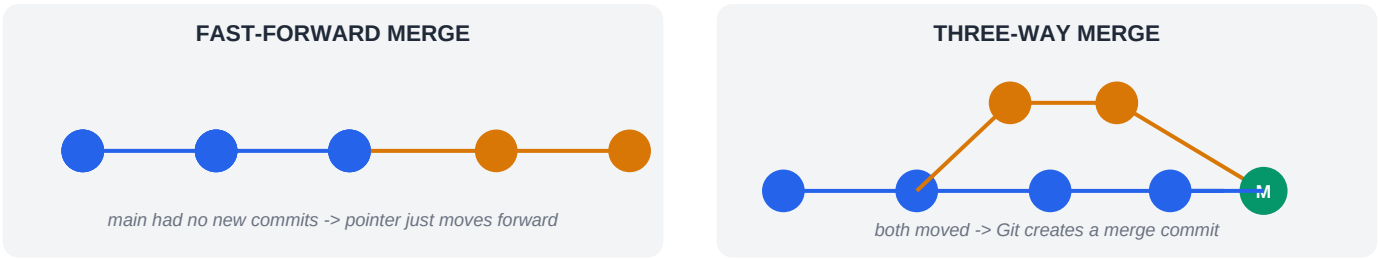


Figure 6 - Two kinds of merge.

11. Merge conflicts

A conflict happens when two branches change the **same lines**. Git stops and marks the file. You decide what the final text should be.

Figure 7 - Conflicts are normal. Resolve them, stage, commit.

```
<<<<<<< HEAD
Hello World        <- your current branch
=======
Hello Git          <- the incoming branch
>>>>>>> feature-login
```

Steps: open the file, keep the right text (or both), delete all three marker lines, then `git add <file>` and `git commit`. In VS Code use **Accept Current / Accept Incoming / Accept Both**.

12. Stashing and tagging

1) git stash 2) switch branch / pull / fix a bug 3) git switch back 4) git stash pop

pop = restore + remove from stash. apply = restore but keep it in stash.

Figure 8 - Stash shelves unfinished work so you can switch tasks.

Tags are permanent labels on a commit, usually for releases like `v1.0.0`. **Lightweight** tags are just a name; **annotated** tags (recommended) store author, date and message.

13. Rebase basics

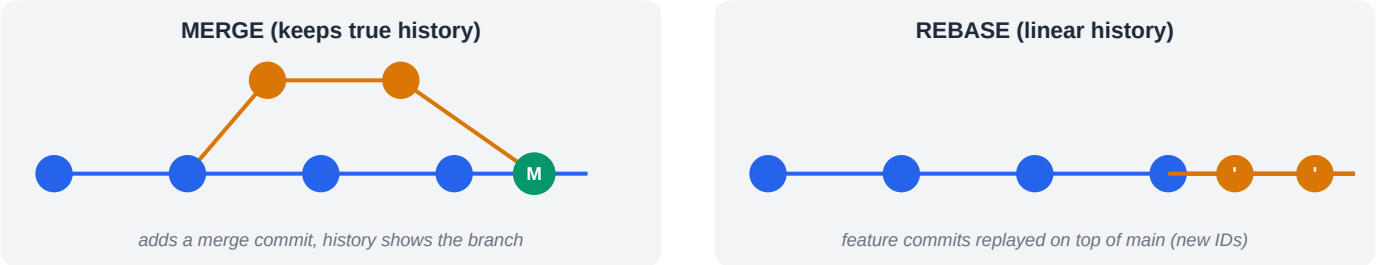
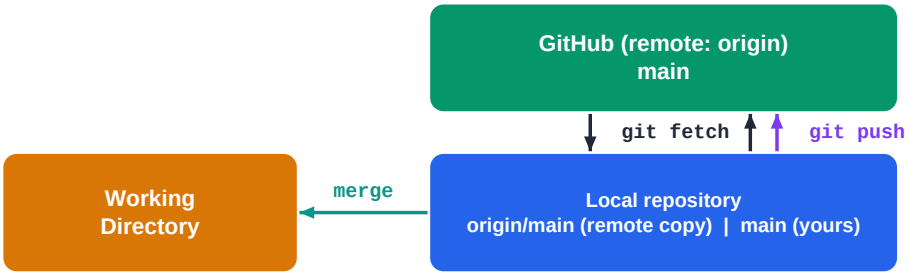


Figure 9 - Merge keeps the branch shape. Rebase replays your commits for a straight line.

> **Golden rule:** never rebase commits that you have already pushed and others use. Rebase only your own local work.

14. Remotes: fetch, pull, push

A **remote** is a repository hosted elsewhere. `origin` is the default nickname for your GitHub URL. `origin/main` is your local record of what GitHub's main looked like at the last fetch.



git pull = git fetch + git merge

fetch: download only, files unchanged
pull: download AND update your files
push: upload your commits

Tip: use fetch to look first, pull to update immediately.

Figure 10 - Fetch downloads, pull downloads and merges, push uploads.

15. GitHub, GitHub Desktop and VS Code

Connect a local project to GitHub: create an empty repository on github.com (**+** then **New repository**, leave README unchecked), copy its URL, then run the remote commands in Part 2. Authentication uses the browser sign-in popup or a Personal Access Token, not your account password.

Task	How
`Clone`	GitHub Desktop: File > Clone repository | VS Code: Command Palette > Git: Clone
`Review changes`	Desktop: Changes tab | VS Code: Source Control (Ctrl+Shift+G), click a file for diff
`Stage`	Desktop: tick checkbox | VS Code: click + beside a file
`Commit`	Write a message, click Commit
`Switch branch`	Desktop: Current Branch menu | VS Code: click branch name, bottom-left
`Pull / Push`	Desktop: Fetch/Pull origin, Push origin | VS Code: Sync Changes
`Publish new project`	VS Code: Source Control > Publish to GitHub

16. The modern workflow (pull requests)

Never commit directly to `main`. Work on a short-lived **feature branch** and merge through a **pull request (PR)** so teammates can review.

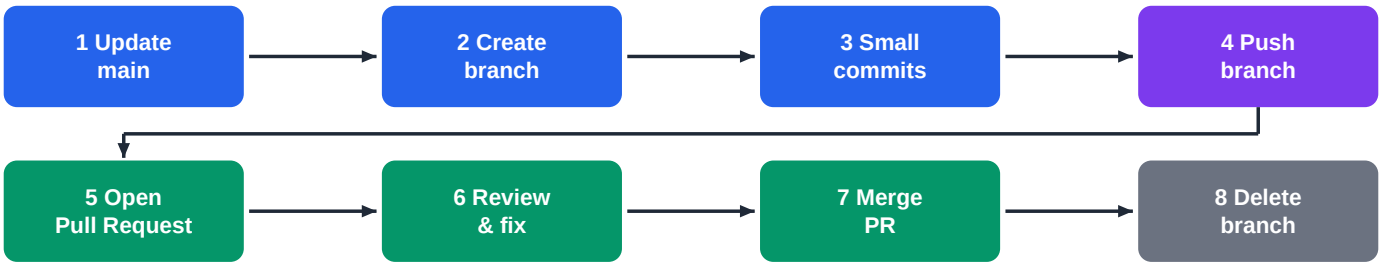


Figure 11 - The professional loop used by most teams.

PART 2 - COMMAND REFERENCE

> `<angle brackets>` mean replace with your own value. `[square brackets]` are optional.

Setup

Command	What it does
`git --version`	Show the installed Git version.
`git config --global user.name "Name"`	Set your name for commits.
`git config --global user.email "e@x.com"`	Set your email for commits.
`git config --global init.defaultBranch main`	New repos start with branch main.
`git config --list`	Show all settings.
`git help <command>`	Open the manual for a command.

Start a repository

Command	What it does
`git init`	Turn the current folder into a Git repository.
`git clone <URL>`	Download a repository from GitHub with full history.
`git clone <URL> <folder>`	Clone into a named folder.

Inspect

Command	What it does
`git status`	Show changed, staged and untracked files. Use often.
`git diff`	Show unstaged changes line by line.
`git diff --staged`	Show what is staged for the next commit.
`git log`	Show commit history.
`git log --oneline --graph --all`	Compact history with a branch graph.
`git show <commit>`	Show one commit in detail.
`git blame <file>`	Show who last changed each line.

Stage and commit

Command	What it does
`git add <file>`	Stage one file.
`git add .`	Stage all changes in the current folder.
`git commit -m "message"`	Save staged changes as a commit.
`git commit -am "message"`	Stage tracked files and commit in one step (not new files).
`git commit --amend`	Fix the last commit message or add forgotten files (local only).
`git rm <file>`	Delete a file and stage the deletion.
`git rm --cached <file>`	Stop tracking a file but keep it on disk.
`git mv <old> <new>`	Rename or move a file.

Undo

Command	What it does
`git restore <file>`	Discard unstaged edits in a file (cannot be undone).
`git restore --staged <file>`	Unstage a file, keep the edits.
`git reset --soft HEAD~1`	Undo the last commit, keep changes staged.
`git reset --hard HEAD~1`	Undo the last commit and delete its changes. Dangerous.
`git revert <commit>`	Make a new commit that reverses an old one. Safe for shared branches.

Branches

Command	What it does
`git branch`	List local branches.
`git branch <name>`	Create a branch without switching.
`git switch <name>`	Move to a branch.
`git switch -c <name>`	Create and switch in one step.
`git branch -d <name>`	Delete a merged branch.
`git branch -D <name>`	Force delete an unmerged branch.
`git branch -M main`	Rename the current branch to main.
`git branch -a`	List local and remote branches.

Merge and conflicts

Command	What it does
`git merge <branch>`	Merge that branch into the current branch.
`git merge --abort`	Cancel a merge that hit conflicts.
`git status`	During a conflict, lists the files that need fixing.
`git add <file>`	Mark a conflict as resolved after editing.
`git commit`	Finish the merge once all conflicts are resolved.

Stash

Command	What it does
`git stash`	Shelve uncommitted changes.
`git stash push -m "msg"`	Stash with a description.
`git stash list`	Show all stashes.
`git stash pop`	Re-apply the newest stash and remove it.
`git stash apply`	Re-apply but keep it in the stash.
`git stash drop`	Delete the newest stash.

Tags

Command	What it does
`git tag`	List tags.
`git tag v1.0.0`	Create a lightweight tag.
`git tag -a v1.0.0 -m "Release 1.0"`	Create an annotated tag.
`git tag -a v0.9 <commit>`	Tag an older commit.
`git push origin v1.0.0`	Upload one tag to GitHub.
`git push --tags`	Upload all tags.
`git tag -d v1.0.0`	Delete a local tag.

Rebase

Command	What it does
`git rebase main`	Replay current branch commits on top of main.
`git rebase --continue`	Continue after fixing a conflict.
`git rebase --abort`	Cancel and return to the original state.

Remotes (GitHub)

Command	What it does
`git remote add origin <URL>`	Link your local repo to GitHub.
`git remote -v`	Show remote URLs.
`git remote set-url origin <URL>`	Change the remote URL.
`git push -u origin main`	First push. -u remembers origin/main.
`git push`	Upload commits to the tracked remote branch.
`git push -u origin <branch>`	Publish a new branch.
`git push origin --delete <branch>`	Delete a remote branch.
`git fetch`	Download new commits without changing your files.
`git pull`	Fetch and merge into the current branch.
`git pull --rebase`	Fetch and rebase instead of merge.

PART 3 - QUICK WORKFLOWS

A. Connect an existing project to GitHub (Git Bash or VS Code terminal)

```
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/your-username/my-project.git
git remote -v
git branch -M main
git push -u origin main
```

Or start from GitHub: create the repo, then `git clone <URL>`, and the remote is already set up.

B. Daily workflow

```
git pull                    # get the latest
# ...edit files...
git status                  # what changed?
git add .
git commit -m "Add login page"
git push
```

C. Full professional pull request workflow

```
git switch main
git pull
git switch -c feature-login

git add .
git commit -m "Add login form"
git add .
git commit -m "Validate email input"

git push -u origin feature-login
# GitHub: Compare & pull request > review > Merge pull request

git switch main
git pull
git branch -d feature-login
git push origin --delete feature-login
```

D. Troubleshooting

Problem	Fix
`remote origin already exists`	Run git remote set-url origin .
`Authentication failed`	Use the browser sign-in popup or a Personal Access Token instead of a password.
`rejected - non-fast-forward`	GitHub has newer commits. Run git pull, fix conflicts, then git push.
`Uncommitted changes block a switch`	Commit them, or run git stash, then switch.
`Committed a secret or big file`	Add it to .gitignore, git rm --cached , commit. Rotate any leaked secret.
`Not a git repository`	You are in the wrong folder, or need git init.
`Detached HEAD`	Run git switch main (or git switch -c to keep the work).

E. Good commit messages

- Short and specific: `Fix navbar overflow`.
- Present tense: Add, Fix, Update.
- One logical change per commit.

F. Final checklist

- [ ] main is up to date (git pull)
- [ ] Working on a feature branch, not main
- [ ] Small commits with clear messages
- [ ] git status checked before committing
- [ ] Secrets and build files are in .gitignore
- [ ] Branch pushed and pull request opened
- [ ] Review feedback addressed, PR merged
- [ ] Branch deleted locally and on GitHub

Developed by Sheharyar Sarmad

LinkedIn: https://www.linkedin.com/in/sheharyar-sarmad-9b7736289/

GitHub: https://github.com/Sheharyar-Sarmad